Shared Collection: Admin-Only Logic — Audit Report

Date: 2026-05-19

Summary - Purpose: document how `isAdminOnly` / admin-only sharing is modeled and enforced across the codebase, list evidence, and surface risks and recommendations.

Key Findings - The admin-only flag is stored on collection/session records and surfaced to the UI via the `StudyCollection` entity. - Client-side checks determine whether a user may modify or re-share an admin-only collection. The check uses group metadata and the local `ChatEntity.isAdmin()` helper. - The persistent backend schema (Appwrite helper) includes boolean columns for `isAdminOnly`, but we found no centralized server-side authorization enforcement in this repository (i.e., rules that reject non-admin writes).

How it works (high level) 1. Data model - `StudyCollection` includes `sharedWithGroupId` and `isAdminOnly` (see StudyCollection.kt). 2. UI / client enforcement - `SharedLibraryView` reads collection metadata and exposes `isAdminOnly` to the UI; import/edit actions are gated in the view layer and by view models (see SharedLibraryView.kt). - The `isAdminOnly` flag is embedded into payloads and used to present read-only or request-access flows to non-admins. 3. Admin determination - Group admin status is resolved by `ChatEntity.isAdmin(userId)` which checks the `adminIds` CSV field (see ChatEntity.kt). 4. Backend schema - Appwrite setup script creates boolean columns `isAdminOnly` for `shared_collections` and `shared_sessions` (see Q_base appwrite/app_write.py).

Evidence & Code References - Data model: StudyCollection.kt - Client UI enforcement: SharedLibraryView.kt - Admin check helper: ChatEntity.kt - Project docs describing behaviour: PROJECT_STRUCTURE_AND_INTEGRATION.md - Appwrite schema columns: Q_base appwrite/app_write.py

Security & Correctness Observations - Enforcement is primarily client-side. If backend write rules are not enforced (Appwrite/Firestore security rules), a malicious client or a compromised user could modify `isAdminOnly` collections or inject content despite UI restrictions. - The admin check relies on a CSV `adminIds` string and a split/contains check — this is simple and effective for local checks but brittle if `adminIds` is ever malformed or not kept in sync. - There’s no explicit cryptographic/authorization binding between the collection metadata and group admin provenance; metadata is distributed via shared documents and wrapped keys (E2EE protects content, not metadata-based permissions).

Recommendations 1. Server-side authorization: enforce admin-only restrictions in the backend (Appwrite/Firestore) so only chat admins can perform writes that modify `isAdminOnly` collections or publish updates to `shared_collections/shared_sessions`. 2. Harden admin identity: store `adminIds` as a structured array rather than CSV and canonicalize comparisons; validate on write operations. 3. Audit & logging: add server-side audit logs for changes to `isAdminOnly` and for grant/rescind operations. 4. Defense-in-depth: continue client checks for UX, but treat client-side checks as advisory only and design server rules to be the source of truth.

Next steps - Optionally I can: (a) scan for any server-side security rules in the repo, (b) create a short PR adding server-side checks or example Appwrite rule snippets, or (c) produce a one-page risk memo for stakeholders.

Prepared by: Qbase codebase audit tooling